

Single-Agent ADK Data Query Agent

Joan Vendrell Gallart

RL Framework for Agent Optimization

We can formulate execution control as a reinforcement learning problem where the agent learns how many reasoning/tool-use steps to allow for each query. The objective is to choose `max_iterations` so that the agent is accurate on multi-filter queries while avoiding unnecessary tool calls on simple queries.

The state combines query features and execution context: $s = (n_f, c_q, a_q, t, b)$. Here, n_f is the number of detected filter fields, a_q is a deterministic flag for unsupported aggregation keywords such as “average,” “how many,” “count,” or “sum,” and t and b are the used and remaining tool-call budgets tracked by the execution loop. The complexity class c_q is derived from these features: one detected filter gives a simple query, multiple filters give a complex query, and aggregation keywords mark an invalid query.

The action is the execution-control parameter selected before running the agent: $a \in \{1, 2, 3, 4, 5\}$, where a corresponds to the chosen value of `max_iterations`. A small value is efficient for simple queries, while a larger value allows sequential filtering for complex queries.

The reward balances correctness and efficiency:

$$r = 1\{\text{correct answer}\} - \lambda \cdot \#\text{tool calls} - \mu \cdot 1\{\text{unsupported query answered}\}, \quad (1)$$

where the first term rewards correct filtering, the second penalizes latency and cost, and the third strongly penalizes unsafe behavior, such as answering an aggregation query that should be declined.

The policy $\pi(a | s)$ maps query features to an execution setting. For this assignment, a simple tabular or small neural policy is sufficient. Training can be performed on a labeled set of valid and invalid queries, where each query is run with different `max_iterations` values and scored using the reward above. The learned policy is then evaluated by measuring accuracy, refusal correctness, average tool calls, and latency.

Note that by defining an external policy $\pi(a | s)$, we are not fine-tuning the LLM decision, but how we interpret it. This design not only simplifies gradient computation

$$\theta \leftarrow \theta + \eta(r - b) \nabla_{\theta} \log \pi_{\theta}(a | s),$$

but also reduces the amount of needed data points to be effectively trained (around 500 – 2,000 query episodes). On the other hand, given more complex task or a faulty model, it may be a requirement to update model parameters too. In that case, we could use similar reward computation as $J(\theta) = \mathbb{E}_{y \sim p_{\theta}(\cdot | q)} [r(y, q)]$. Such that its gradient would be computed as

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[(r - b) \sum_{i=1}^k \nabla_{\theta} \log p_{\theta}(y_i | q, y_{<i}) \right],$$

requiring from more data points to be effectively trained (around 10,000 – 100,000 query episodes).

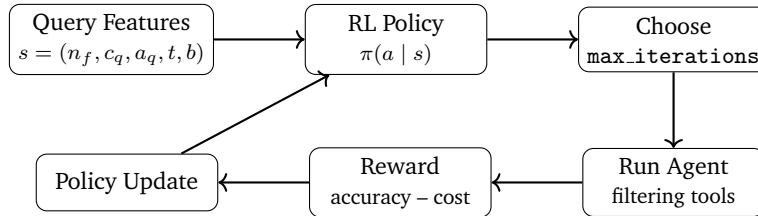


Figure 1: RL framework for automatically selecting execution-control parameters.